Travlr Getaways

CS 465 Project Software Design Document

Version 1.0

Table of Contents

CS 465 Project Software Design Document 1
Table of Contents 2
Document Revision History 2
Instructions 2
Executive Summary 3
Design Constraints 3
System Architecture View 3
Component Diagram 3
Sequence Diagram 4
Class Diagram 4
API Endpoints 4
The User Interface 4

Document Revision History

Version	Date	Author	Comments
1.0	01/22/2026	Adrian Tull	Updated static html pages to include Handlbars for dynamic page generation
1.1	02/08/2026	Adrian Tull	Updated sequence and class diagrams

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

<Describe the appropriate architecture of the web application based on your client's software requirements. Be sure to reference your use of the MEAN stack for development. Explain both the customer-facing side of the application and the administrator single-page application (SPA).>

The Travlr Getaways web app is built using the MEAN stack (MongoDB, Express.js, Angular, and Node.js). This architecture separates the app into two parts.

1. Customer Facing: a server rendered Express application using Handlebars templating. This makes a multipage navigation where each page request is processed by the server, rendered with data, and returned as complete HTML. This approach is SEO friendly and works well for pages with a lot of content like the trip listings, rooms, meals, etc.
2. Admin SPA: An Angular single page application that gives an interactive experience that goes further than static HTML can provide for managing trips and content. The SPA should load once and dynamically updates content through API calls making interactions faster for admin tasks like creating, editing, and deleting trips without having to do full page reloads

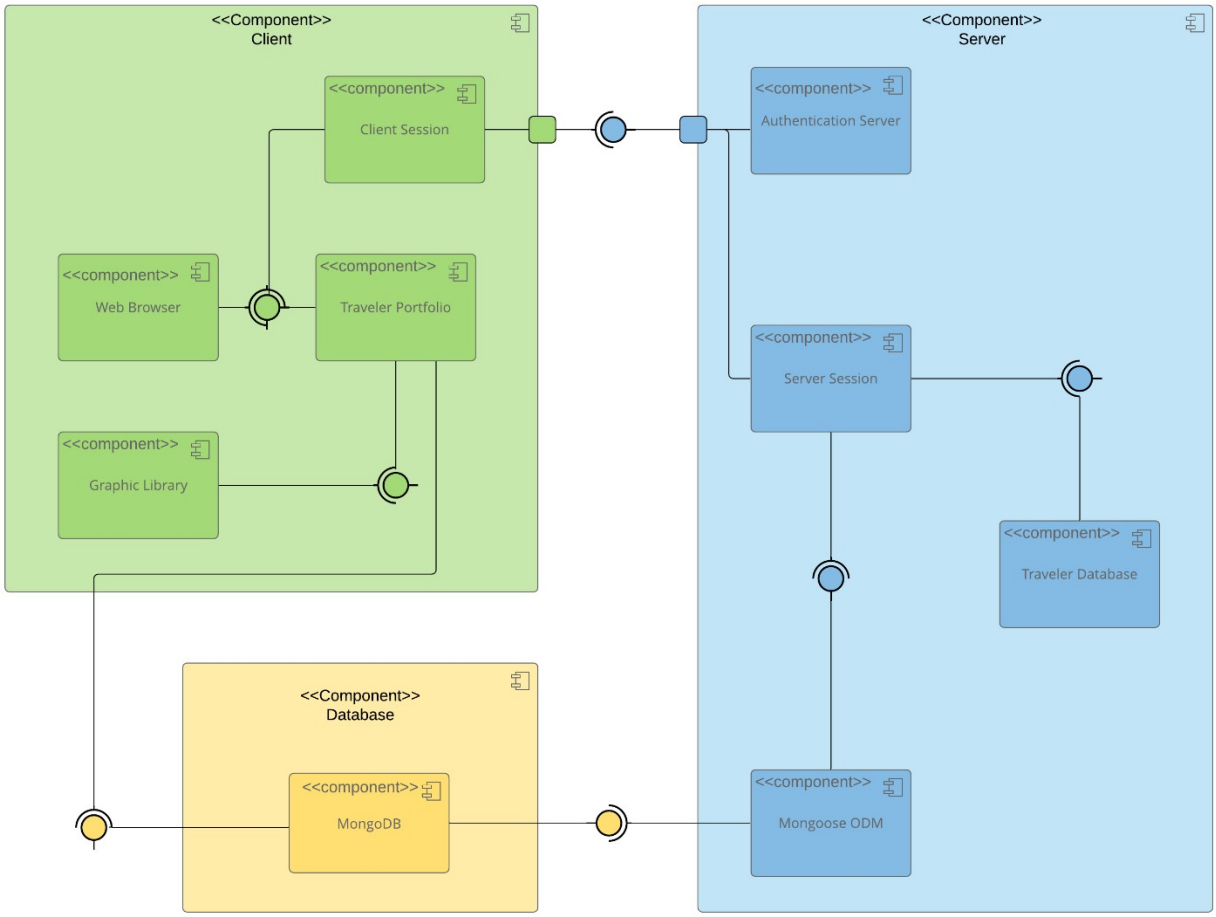
Design Constraints

<Identify the design constraints for developing the web-based Travlr Getaways application. Explain the implications of the design constraints on the application development.>

1. Node.js: server has to run on Node with Express
2. Handlebars Templating: views use .hbs files with partials for reusable header and footer components
3. MVC: the application follows the Model View Controller separation with dedicated folders for routes, controllers, and views.
4. Data Storage: trip data, which we are storing in the trips.json, is really not how anyone would build a website as we would have to recommit and rebuild our website every time content changed, but that is currently the constraint we have and we will presumably be moving to MongoDB to hold that as well as the rest of the content such as images. This would eventually leave the website as a frame for which we can fill with content.

System Architecture View

Component Diagram



A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version].

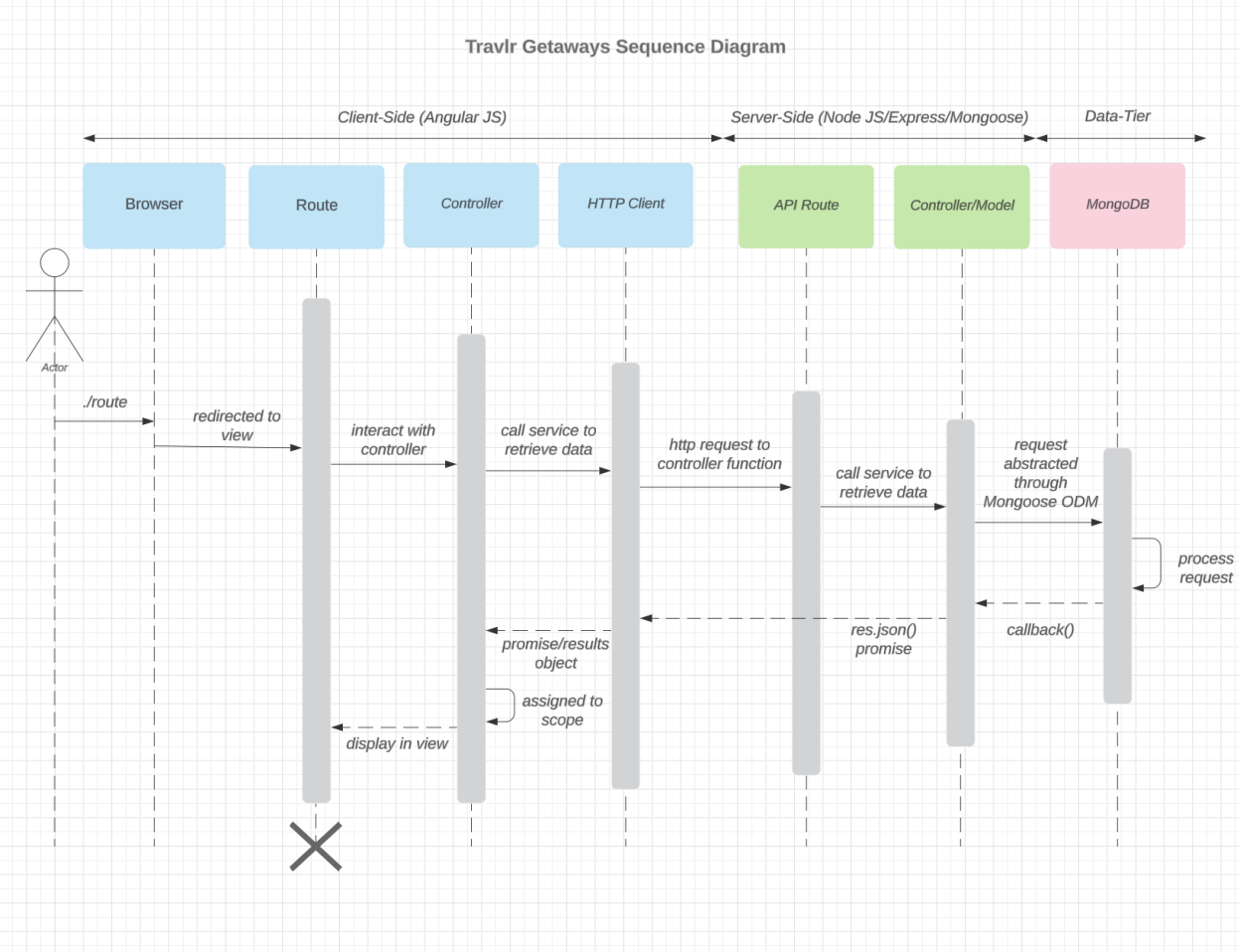
<Describe the overall system architecture of the web application by referring to the component diagram above. Identify the significant components that will be used and their relationships to one another.>

The Travlr Getaways web app uses separation of concerns with its architectural approach. We separate ui, server side logic, and data persistence with the database. The client represents the user's environment. The Traveler portfolio handles the display of travel content while the graphic library renders the visual components on the site. The client session will manage the local state and is the component that connects the user's experience to the server. The traveler portfolio acts as the connection from the user to the database. In a production setting, this usually not be how I would implement this connection as it could potentially be a way for the user to execute arbitrary queries on the database. When building a system with sensitive data, like the admin would likely have or a customer purchasing one of the trips, you would first want to validate and clean any request and reject any malformed request to avoid unfettered access to the db. The db can connect to the server through the Mongoose ODM which is running on the server to query things such as doing our standard CRUD operations to the traveler content.

•

Sequence Diagram

<Illustrate the flow of logic in a web application by completing a sequence diagram. Insert an image of the sequence diagram here.>



The flow starts when a user goes to TravlrGetaways route in the browser. The client side view layer receives the request and resolves the template. The route hands off to the controller, which determines that it needs data and makes an HTTP request to the server side API from the HTTP client. The API route picks up that request and forwards it to the controller/model layer where mongoose handles the interaction with MongoDB. Once the db returns the requested data, the response goes back through each layer in revers order, and then the result is rendered onto the screen for the user.

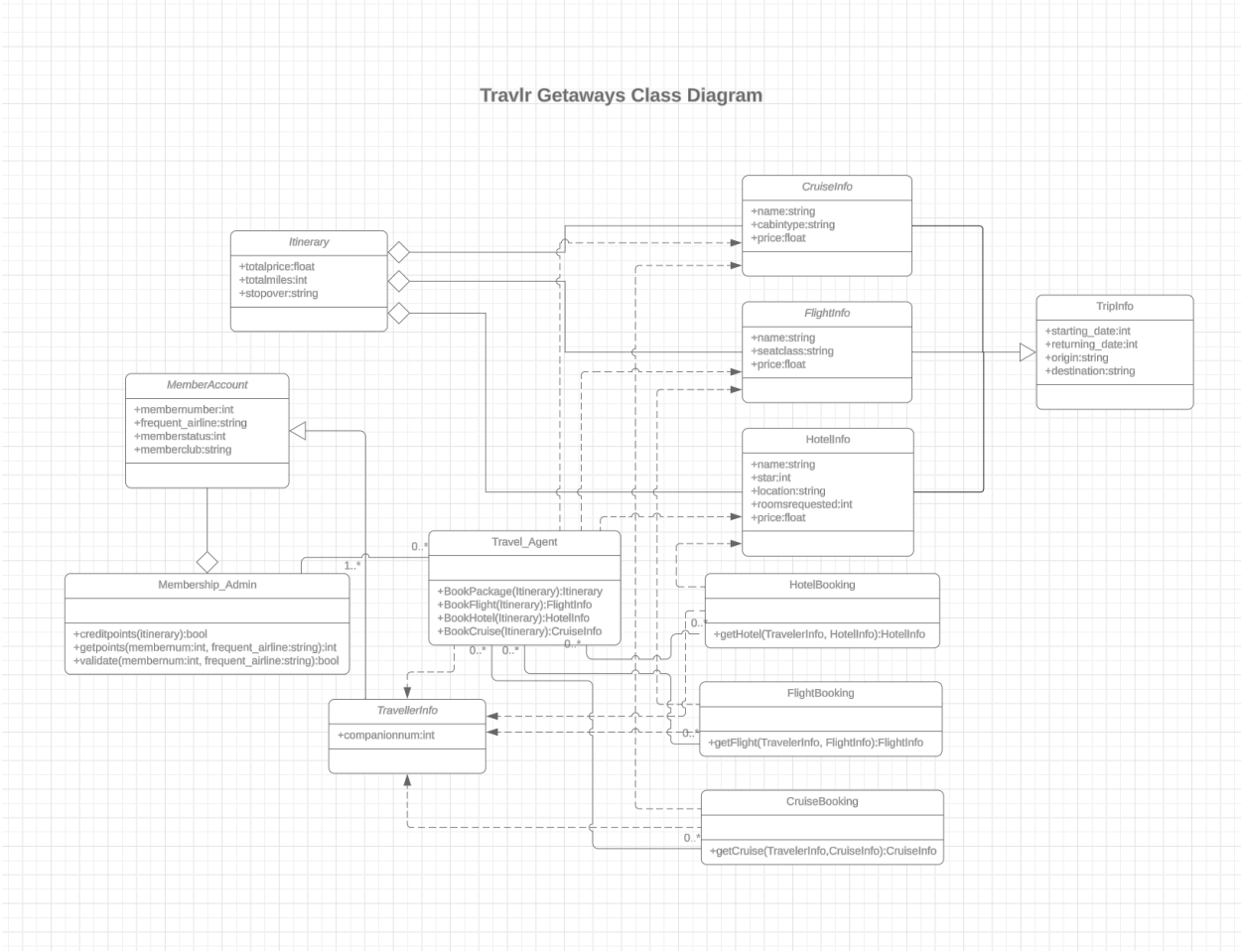
When a trips request hits the api/trips and returns the trip listings, a sign in submits credentials to an authentication endpoint for validation against the user data in the MongoDB database, and Admin interactions send POST or PUT requests to create or update trip records. These all follow the same client to server to database cycle with each layer handling its own responsibility.

<Describe the flow of logic in the web application based on the sequence diagram. Be sure to describe the interactions between the layers, or tiers, of the full stack application. It will be helpful to include significant processes such as Sign In, Trips, and Admin interactions when referring to the sequence diagram.>

Class Diagram

<Illustrate the JavaScript classes of the web application by completing a class diagram for the web application. Insert an image of the class diagram here.>

<Describe the JavaScript classes of the web application based on the class diagram.>



- Travel_Agent
  - serves as the central booking interface, exposing methods for packages, flights, hotels, and cruises that each accept an Itinerary object. It has a composition relationship with Membership_Admin and dependencies on the booking and traveller classes.
- Itinerary
  - aggregates CruiseInfo, FlightInfo, and HotelInfo to represent a complete travel plan, tracking total price, total miles, and stopover information.
- CruiseInfo
  - stores specific details for the cruise including the cruise name, cabin type, and price.
- FlightInfo
  - stores specific details for the flight including the flight name, seat class, and price.
- HotelInfo

    - stores specific details for the hotel including the hotel name, star rating, location, number of rooms requested, and price.
- HotelBooking
    - depends on TravellerInfo and HotelInfo to retrieve finalized hotel booking details.
- FlightBooking
    - depends on TravellerInfo and FlightInfo to retrieve finalized flight booking details
- CruiseBooking
    - depends on TravellerInfo and CruiseInfo to retrieve finalized cruise booking details
- TravelerInfo
    - holds companion data and is referenced by each of the three booking classes.
- TripInfo
    - captures trip logistics including departure and return dates, origin, and destination, and maintains an association with TravellerInfo.
- MemberAccount
    - stores member-level data such as member number, frequent airline, member status, and club affiliation.
- Membership_Admin
    - extends MemberAccount with administrative operations for crediting points, retrieving point balances, and validating members. It holds a composition relationship with Travel_Agent, owning and managing one or more agent instances.

API Endpoints

<Exposing RESTful endpoints is a design approach to enable an application to participate in a larger ecosystem. Document each endpoint in the table below, including the HTTP method, purpose, URL, and notes.>

Method	Purpose	URL	Notes
GET	Get all trips	/api/trips	RESTful API endpoint; queries MongoDB via Mongoose Trip model using Model.find() and returns a JSON array of all trip documents
GET	Get a specific trip from the trip code	/api/trips/:tripcode	RESTful API endpoint; accepts a tripCodeURL parameter, queries MongoDB using Model.find({ 'code': tripCode }), and returns a json array filtered by the trip code

The User Interface

<Insert screenshots from the development of the SPA development to show the following: (1) a unique trip, added by you, (2) the Edit screen, and (3) the Update screen.>

<Summarize the Angular project structure and how it compares to the Express project structure. Be sure to describe the rich functionality provided by the SPA compared to a simple web application interaction. Describe the process of testing to make sure the SPA is working with the API to GET and PUT data in the database.>

The project juxtaposes an Angular SPA with an Express based, server side architecture. The Express structure uses separation of concerns with a RESTful API for data and a server for rendering static Handlebars templates, the Angular framework moves presentation logic to the client using modular components and injectible services. The SPA makes it possible to make changes to the page without a full page refresh letting the user have faster view transitions and dynamic interactivity. While testing, I made sure that the Angular TripDataService interfaces with the Express backend. I verified that GET requests accurately retrieved trip payloads for display and the PUT methods successfully made changes based on the user edits to the MongoDB database.